CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

cspit
Chandubhai S Patel Institute of Technology

Faculty of Technology and Engineering

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

Date: 13/10/2026

Laboratory Manual

Academic Year	:	2026-27	Semester	:	3rd
Course code	:	ITUE203	Course name	:	WEB DEVELOPMENT FRAMEWORKS

Practical – 13

PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem Statement:- Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

Objective:- To design and implement a lightweight, REST-like JSON API using PHP to manage student data, and to build a dynamic, single-page user interface that consumes these endpoints using the JavaScript Fetch API to perform asynchronous CRUD (Create, Read, Update, Delete) operations without full-page reloads.

db.php:

```php
<?php
// db.php - MySQL connection (PDO)
$DB_HOST = 'localhost';
$DB_NAME = 'student_api_db';
$DB_USER = 'root';   // change if needed
$DB_PASS = '';       // XAMPP default is empty

try {
    $pdo = new PDO(
        "mysql:host=$DB_HOST;dbname=$DB_NAME;charset=utf8mb4",
        $DB_USER,
        $DB_PASS,
        [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ]
    );
} catch (PDOException $e) {
    http_response_code(500);
    header('Content-Type: application/json');
    echo json_encode(['success' => false, 'message' => 'Database connection failed']);
    exit;
}
```

index.html:

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Manager - PHP JSON API + Fetch</title>
  <style>
    * { box-sizing: border-box; }
```

```
    body { font-family: Arial, sans-serif; background: #f4f6f9; margin: 0;
padding: 20px; color: #222; }
    .container { max-width: 900px; margin: auto; }
    h1 { text-align: center; color: #1f3a8a; }
    .card { background: #fff; padding: 20px; border-radius: 8px; box-shadow: 0
2px 6px rgba(0,0,0,.1); margin-bottom: 20px; }
    form { display: grid; grid-template-columns: repeat(auto-fit,
minmax(180px, 1fr)); gap: 10px; }
    input { padding: 10px; border: 1px solid #ccc; border-radius: 4px; font-
size: 14px; }
    button { padding: 10px 14px; border: 0; border-radius: 4px; cursor:
pointer; font-size: 14px; color: #fff; }
    .btn-primary { background: #2563eb; }
    .btn-secondary { background: #6b7280; }
    .btn-edit { background: #f59e0b; }
    .btn-delete { background: #dc2626; }
    button:disabled { opacity: .6; cursor: not-allowed; }
    table { width: 100%; border-collapse: collapse; }
    th, td { padding: 10px; border-bottom: 1px solid #e5e7eb; text-align:
left; }
    th { background: #eef2ff; }
    td.actions { display: flex; gap: 6px; }
    #message { padding: 10px; border-radius: 4px; margin-bottom: 15px;
display: none; }
    #message.success { display: block; background: #dcfce7; color: #166534; }
    #message.error { display: block; background: #fee2e2; color: #991b1b; }
    #loader { display: none; text-align: center; padding: 15px; color:
#2563eb; font-weight: bold; }
    .spinner { display: inline-block; width: 16px; height: 16px; border: 3px
solid #bfdbfe;
               border-top-color: #2563eb; border-radius: 50%; animation: spin
.8s linear infinite; vertical-align: middle; margin-right: 8px; }
    @keyframes spin { to { transform: rotate(360deg); } }
    .empty { text-align: center; color: #6b7280; padding: 20px; }
  </style>
</head>
<body>
  <div class="container">
    <h1>Student Manager</h1>

    <div id="message"></div>

    <div class="card">
      <h3 id="formTitle">Add Student</h3>
      <form id="studentForm">
        <input type="hidden" id="studentId">
        <input type="text" id="name" placeholder="Full name" required>
        <input type="email" id="email" placeholder="Email" required>
```

```
        <input type="text" id="course" placeholder="Course" required>
        <button type="submit" class="btn-primary" id="submitBtn">Add</button>
        <button type="button" class="btn-secondary" id="cancelBtn"
style="display:none;">Cancel</button>
      </form>
    </div>

    <div class="card">
      <h3>Student List</h3>
      <div id="loader"><span class="spinner"></span>Loading...</div>
      <table id="studentTable">
        <thead>
          <tr><th>ID</th><th>Name</th><th>Email</th><th>Course</th><th>Actions
</th></tr>
        </thead>
        <tbody id="studentBody"></tbody>
      </table>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

Api.php

```
<?php
// api.php - JSON API for students (list, add, update, delete)
header('Content-Type: application/json; charset=utf-8');
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit;
}

require_once 'db.php';

function respond($data, $code = 200) {
    http_response_code($code);
    echo json_encode($data);
    exit;
}

// Optional artificial delay (ms) to demo the loading state: api.php?delay=800
if (isset($_GET['delay'])) {
    usleep(min((int)$_GET['delay'], 3000) * 1000);
```

```
}

$method = $_SERVER['REQUEST_METHOD'];
$input  = json_decode(file_get_contents('php://input'), true) ?? [];

function validate($d) {
    $name   = trim($d['name']   ?? '');
    $email  = trim($d['email']  ?? '');
    $course = trim($d['course'] ?? '');
    if ($name === '' || $email === '' || $course === '') {
        respond(['success' => false, 'message' => 'Name, email and course are
required'], 400);
    }
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        respond(['success' => false, 'message' => 'Invalid email address'],
400);
    }
    return [$name, $email, $course];
}

try {
    switch ($method) {

        // LIST (all) or single record with ?id=
        case 'GET':
            if (isset($_GET['id'])) {
                $stmt = $pdo->prepare('SELECT * FROM students WHERE id = ?');
                $stmt->execute([(int)$_GET['id']]);
                $row = $stmt->fetch();
                if (!$row) respond(['success' => false, 'message' => 'Student
not found'], 404);
                respond(['success' => true, 'data' => $row]);
            }
            $rows = $pdo->query('SELECT * FROM students ORDER BY id DESC')-
>fetchAll();
            respond(['success' => true, 'data' => $rows]);
            break;

        // ADD
        case 'POST':
            [$name, $email, $course] = validate($input);
            $stmt = $pdo->prepare('INSERT INTO students (name, email, course)
VALUES (?, ?, ?)');
            $stmt->execute([$name, $email, $course]);
            respond([
                'success' => true,
                'message' => 'Student added',
                'id'      => (int)$pdo->lastInsertId(),
```

```
            ], 201);
            break;

        // UPDATE
        case 'PUT':
            $id = (int)($input['id'] ?? 0);
            if ($id <= 0) respond(['success' => false, 'message' => 'Valid id
is required'], 400);
            [$name, $email, $course] = validate($input);
            $stmt = $pdo->prepare('UPDATE students SET name = ?, email = ?,
course = ? WHERE id = ?');
            $stmt->execute([$name, $email, $course, $id]);
            respond(['success' => true, 'message' => 'Student updated']);
            break;

        // DELETE
        case 'DELETE':
            $id = (int)($input['id'] ?? ($_GET['id'] ?? 0));
            if ($id <= 0) respond(['success' => false, 'message' => 'Valid id
is required'], 400);
            $stmt = $pdo->prepare('DELETE FROM students WHERE id = ?');
            $stmt->execute([$id]);
            if ($stmt->rowCount() === 0) {
                respond(['success' => false, 'message' => 'Student not
found'], 404);
            }
            respond(['success' => true, 'message' => 'Student deleted']);
            break;

        default:
            respond(['success' => false, 'message' => 'Method not allowed'],
405);
    }
} catch (PDOException $e) {
    // 1062 = duplicate entry (unique email)
    if ($e->errorInfo[1] == 1062) {
        respond(['success' => false, 'message' => 'Email already exists'],
409);
    }
    respond(['success' => false, 'message' => 'Server error'], 500);
}
```

Schema.sql

```
CREATE DATABASE IF NOT EXISTS student_api_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE student_api_db;
```

```sql
CREATE TABLE IF NOT EXISTS students (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  email      VARCHAR(150) NOT NULL UNIQUE,
  course     VARCHAR(100) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO students (name, email, course) VALUES
  ('Aarav Patel', 'aarav@example.com', 'B.Tech IT'),
  ('Diya Shah',   'diya@example.com',  'B.Tech CE');
```

Script.js

```javascript
// script.js - consumes api.php using the Fetch API (no page reloads)
const API_URL = 'api.php';

const form        = document.getElementById('studentForm');
const idInput     = document.getElementById('studentId');
const nameInput   = document.getElementById('name');
const emailInput  = document.getElementById('email');
const courseInput = document.getElementById('course');
const submitBtn   = document.getElementById('submitBtn');
const cancelBtn   = document.getElementById('cancelBtn');
const formTitle   = document.getElementById('formTitle');
const tbody       = document.getElementById('studentBody');
const loader      = document.getElementById('loader');
const messageBox  = document.getElementById('message');

let students = [];

// ---------- helpers ----------
function showMessage(text, type = 'success') {
  messageBox.textContent = text;
  messageBox.className = type;
  setTimeout(() => { messageBox.className = ''; messageBox.textContent = '';
}, 3500);
}

function setLoading(isLoading) {
  loader.style.display = isLoading ? 'block' : 'none';
}

function escapeHTML(str) {
  const div = document.createElement('div');
  div.textContent = str;
  return div.innerHTML;
```

```
}

// Generic fetch wrapper: parses JSON and throws on non-OK responses
async function request(url, options = {}) {
  const res = await fetch(url, {
    headers: { 'Content-Type': 'application/json' },
    ...options,
  });
  const data = await res.json().catch(() => ({}));
  if (!res.ok || data.success === false) {
    throw new Error(data.message || `Request failed (${res.status})`);
  }
  return data;
}

// ---------- LIST ----------
async function loadStudents() {
  setLoading(true);
  try {
    const result = await request(API_URL);   // GET
    students = result.data;
    renderTable();
  } catch (err) {
    tbody.innerHTML = `<tr><td colspan="5" class="empty">Could not load
students.</td></tr>`;
    showMessage(err.message, 'error');
  } finally {
    setLoading(false);
  }
}

function renderTable() {
  if (students.length === 0) {
    tbody.innerHTML = `<tr><td colspan="5" class="empty">No students
found.</td></tr>`;
    return;
  }
  tbody.innerHTML = students.map(s => `
    <tr>
      <td>${s.id}</td>
      <td>${escapeHTML(s.name)}</td>
      <td>${escapeHTML(s.email)}</td>
      <td>${escapeHTML(s.course)}</td>
      <td class="actions">
        <button class="btn-edit" onclick="editStudent(${s.id})">Edit</button>
        <button class="btn-delete"
onclick="deleteStudent(${s.id})">Delete</button>
      </td>
```

```
    </tr>
  `).join('');
}

// ---------- ADD / UPDATE ----------
form.addEventListener('submit', async (e) => {
  e.preventDefault();

  const payload = {
    name:   nameInput.value.trim(),
    email:  emailInput.value.trim(),
    course: courseInput.value.trim(),
  };
  const id = idInput.value;

  submitBtn.disabled = true;
  try {
    if (id) {
      payload.id = Number(id);
      await request(API_URL, { method: 'PUT', body: JSON.stringify(payload)
});
      showMessage('Student updated successfully');
    } else {
      await request(API_URL, { method: 'POST', body: JSON.stringify(payload)
});
      showMessage('Student added successfully');
    }
    resetForm();
    await loadStudents();
  } catch (err) {
    showMessage(err.message, 'error');
  } finally {
    submitBtn.disabled = false;
  }
});

// ---------- EDIT (fill form) ----------
function editStudent(id) {
  const s = students.find(st => Number(st.id) === Number(id));
  if (!s) return;
  idInput.value     = s.id;
  nameInput.value   = s.name;
  emailInput.value  = s.email;
  courseInput.value = s.course;
  formTitle.textContent = 'Edit Student';
  submitBtn.textContent = 'Update';
  cancelBtn.style.display = 'inline-block';
  window.scrollTo({ top: 0, behavior: 'smooth' });
```

```
}

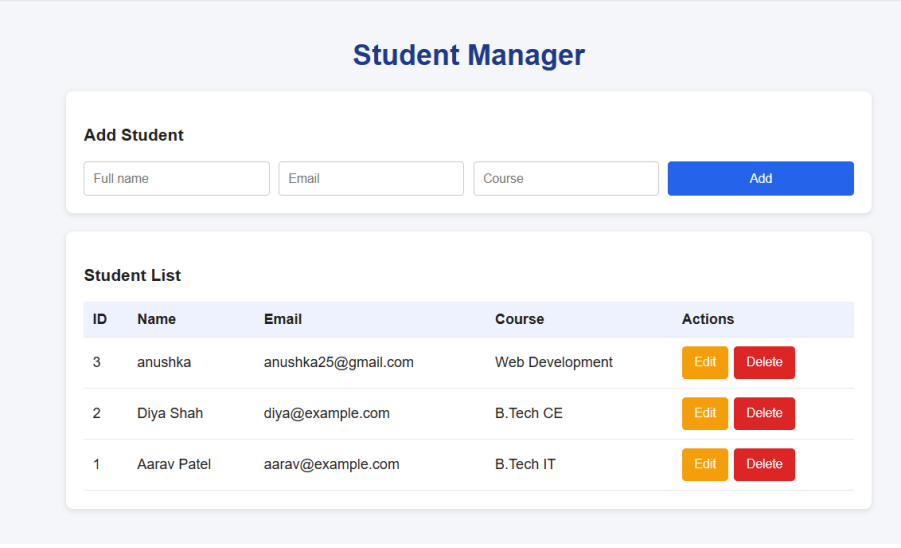
function resetForm() {
  form.reset();
  idInput.value = '';
  formTitle.textContent = 'Add Student';
  submitBtn.textContent = 'Add';
  cancelBtn.style.display = 'none';
}
cancelBtn.addEventListener('click', resetForm);

// ---------- DELETE ----------
async function deleteStudent(id) {
  if (!confirm('Are you sure you want to delete this student?')) return;
  try {
    await request(API_URL, { method: 'DELETE', body: JSON.stringify({ id })
});
    showMessage('Student deleted successfully');
    await loadStudents();
  } catch (err) {
    showMessage(err.message, 'error');
  }
}

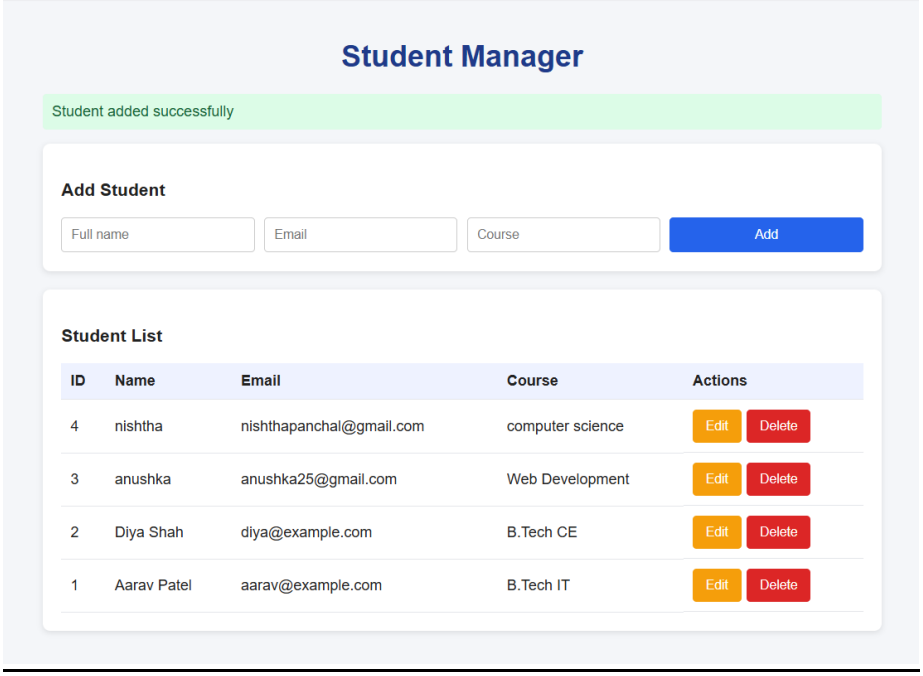
// Initial load
loadStudents();
```

UI SCREENSHOTS:

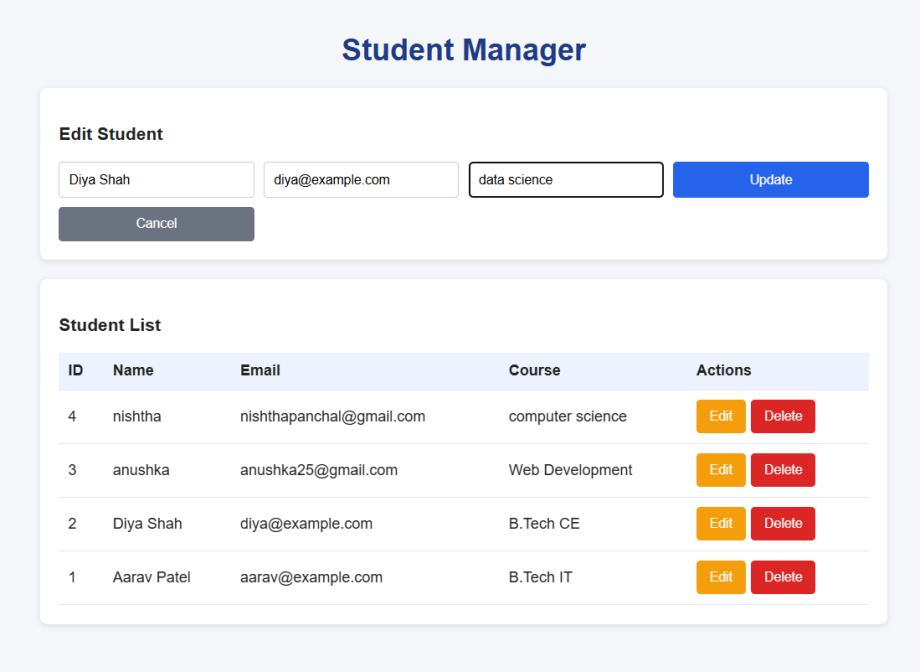
List:



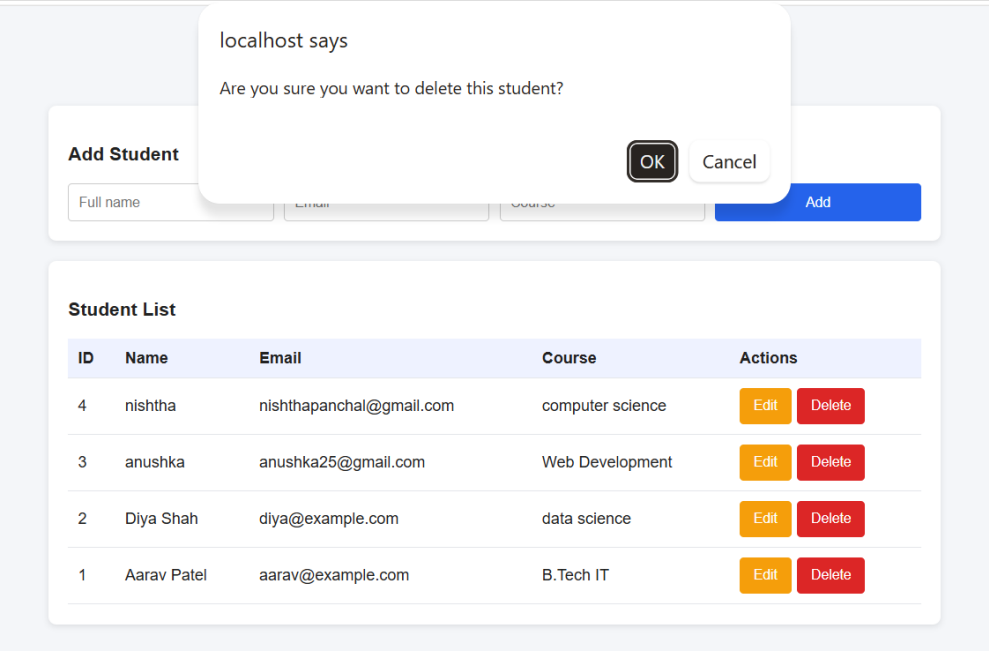
Add:


Student Manager
Student added successfully
Add Student
Full name
Email
Course
Add
Student List
ID Name Email Course Actions
4 nishtha nishthapanchal@gmail.com computer science Edit Delete
3 anushka anushka25@gmail.com Web Development Edit Delete
2 Diya Shah diya@example.com B.Tech CE Edit Delete
1 Aarav Patel aarav@example.com B.Tech IT Edit Delete

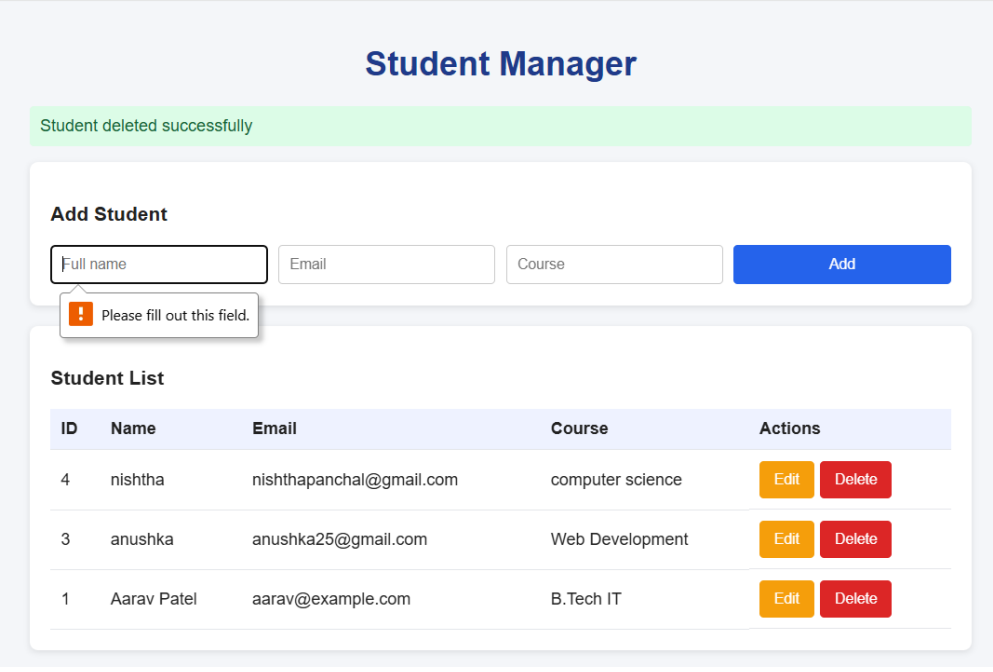
Edit:


Student Manager
Edit Student
Diya Shah
diya@example.com
data science
Update
Cancel
Student List
ID Name Email Course Actions
4 nishtha nishthapanchal@gmail.com computer science Edit Delete
3 anushka anushka25@gmail.com Web Development Edit Delete
2 Diya Shah diya@example.com B.Tech CE Edit Delete
1 Aarav Patel aarav@example.com B.Tech IT Edit Delete

Delete:



Error/Loading states:

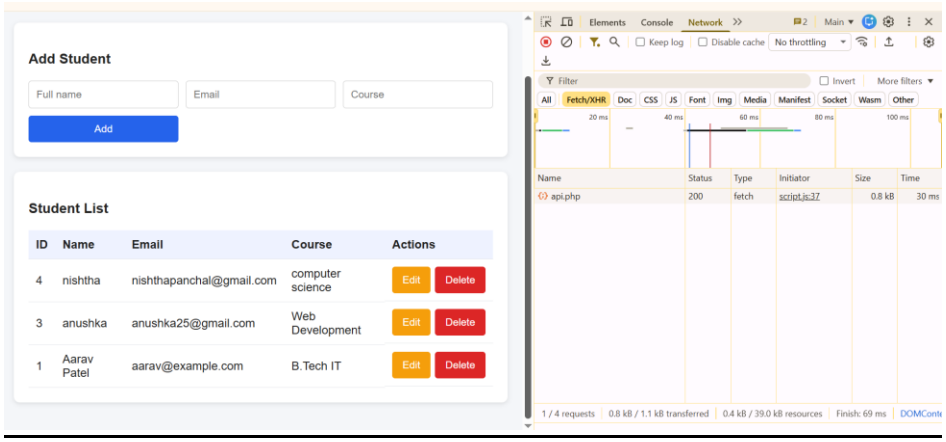


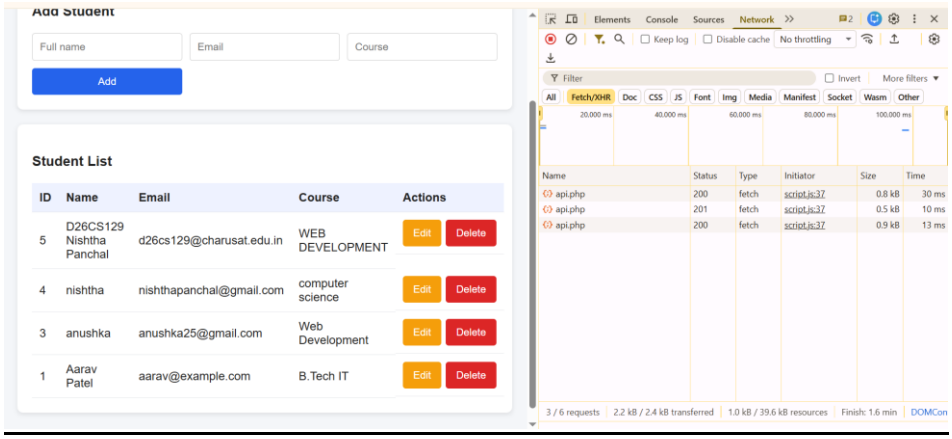
NETWORK SCREENSHOTS:

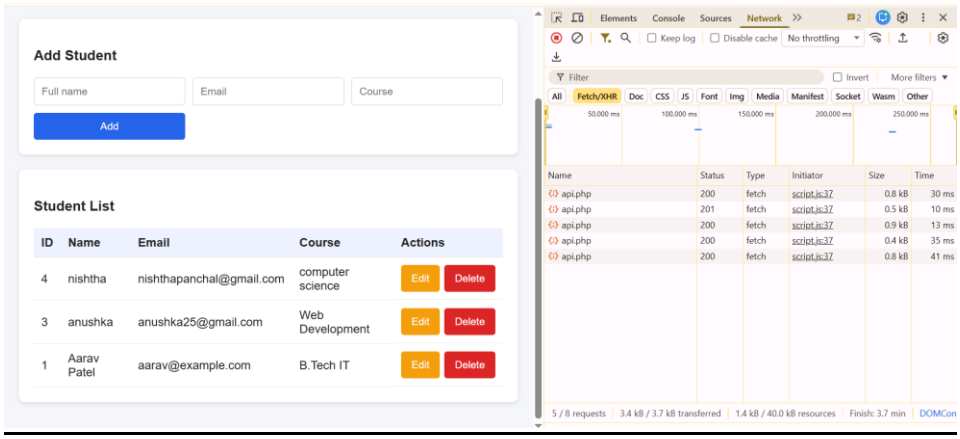
Browser Network tab showing Fetch requests:

Status code

JSON responses







Learning Outcome:

After completing this practical, I am able to:

1. **Build a PHP JSON API:** create endpoints in PHP that handle list, add, update and delete operations and return responses in JSON format.
2. **Use the Fetch API:** send asynchronous GET, POST, PUT and DELETE requests from JavaScript to a server using `fetch()` with `async/await`.
3. **Update the page without reloading:** change the DOM dynamically, so the student list refreshes without a full page reload.
4. **Connect PHP to MySQL securely:** use PDO and prepared statements for database operations, which protect against SQL injection.
5. **Handle HTTP status codes and errors:** work with codes such as 200, 201, 400, 404, 409 and 500, and show success, error and loading messages to the user.
6. **Debug with browser DevTools:** inspect requests, status codes and JSON responses in the Network tab.

S